NS3SIONNA: MIMO-ABSTRAKTIONSMODELL FÜR IEEE 802.11AX/BE

Stand: 18. Juli 2026

Arbeitszweig: feature/su-mimo-minimal

Basis: ns-3.40, ns3sionna c896b858, Sionna 1.2.1

1. ZIEL UND ABGRENZUNG

Sionna RT berechnet die komplexe, frequenzselektive
Kanalübertragungsmatrix. Die eigentliche Link-Abstraktion, also
Post-Processing-SINR, Effective-SINR, MCS/PER und später die Auswahl von
Precoding und MU-MIMO, wird ausschließlich in ns-3 ausgeführt.

Die Entwicklung erfolgt in kleinen, einzeln testbaren Schritten. Der aktuelle
Stand implementiert Unterphase 1.1: einen dimensionssicheren CFR-Container und
den deterministischen numerischen Kern für ein minimales SU-MIMO-Modell. Noch
nicht enthalten sind der MIMO-Transport vom Python-Server, eine
802.11ax-Codierungs-/PER-Abbildung und die Einkopplung in WifiPhy.

2. REPRODUZIERBARE AUSGANGSBASIS

Die lokale Referenzplattform ist CPU-only:

- Ubuntu 24.04, GCC 13.3, CMake 3.28
- ns-3.40 mit Tests und Beispielen
- ZeroMQ 4.3.5 und Protocol Buffers 3.21.12
- Python 3.12.12, Sionna 1.2.1, NumPy 1.26.4
- TensorFlow erkennt keine GPU und verwendet den CPU-Pfad

Der unveränderte Server setzt zurzeit jeweils eine 1x1-Sende- und
Empfangsantenne. Seine normalisierte CFR erfüllt

$$
\frac{1}{N_\mathrm{sc}}\sum_k |h[k]|^2 = 1,
$$

während der Wideband-Pfadverlust separat in dB übertragen wird. Diese Trennung
wird für MIMO beibehalten: H[k] enthält Small-Scale-Fading und der skalare
Wideband-Verlust bestimmt die Referenz-SNR.

3. ENTWICKLUNGSPLAN

PHASE 0: BASELINE UND MESSVERTRAG

1. Versionen und CPU-Ausführung reproduzierbar festhalten.
2. Bestehende SISO-Import-, Unit- und Beispieltests ausführen.
3. Einheiten, Antennenreihenfolge, CFR-Normalisierung und
   Subcarrier-Reihenfolge als Datenvertrag festlegen.

Abnahmekriterium: SISO-Verhalten bleibt unverändert und alle alten Tests laufen.

PHASE 1: BELASTBARES SU-MIMO-MINIMALMODELL

UNTERPHASE 1.1: NUMERISCHER KERN (IMPLEMENTIERT)

- CFR-Tensor H[k] mit expliziten Dimensionen N_sc x N_rx x N_tx.
- Open-Loop Spatial Multiplexing, ein Stream je Sendeantenne.
- Gleiche Gesamtleistungsaufteilung über N_s = N_tx Streams.
- Perfekte CFR-Kenntnis am Empfänger und linearer MMSE-Detektor.

- Post-MMSE-SINR pro Subcarrier und Stream.
- Mutual-Information-basierte Kompression über Subcarrier.
- Analytische Unit-Tests für SISO, orthogonale 2x2-Kanäle und frequenzselektive Kanäle.

Abnahmekriterium: analytische Werte werden mit einer Toleranz von 1e-10 reproduziert.

UNTERPHASE 1.2: MIMO-DATENVERTRAG SIONNA -> NS-3

- Antennendimensionen rückwärtskompatibel in message.proto ergänzen.
- Flache Reihenfolge sc/rx/tx verbindlich festlegen.
- Sionna-Arrays konfigurierbar machen und die CFR ohne Verlust der Antennendimensionen serialisieren.
- CacheEntry auf MimoChannelMatrix umstellen.
- Python-Shape-Tests und C++-Deserialisierungstests hinzufügen.

Abnahmekriterium: 1x1 ist binär/funktional kompatibel; synthetische 2x2-Werte kommen indexgenau in ns-3 an.

UNTERPHASE 1.3: 802.11AX-LINK-ABSTRAKTION

- Referenz-SNR aus Sendeleistung, Wideband-Verlust, Bandbreite, Rauschzahl und thermischem Rauschen berechnen.
- Effektive SINR zunächst mit einer explizit kalibrierbaren EESM- oder MIESM-Abbildung je MCS bestimmen.
- Vorhandene ns-3 ErrorRateModel-Schnittstellen soweit möglich wiederverwenden.
- SISO-Parität und AWGN-Kurven gegen ns-3 validieren.

Abnahmekriterium: PER-vs-SNR-Kurven sind monoton, reproduzierbar und im SISO-Grenzfall deckungsgleich zur festgelegten Referenz.

UNTERPHASE 1.4: WIFIPHY-INTEGRATION UND BEISPIEL

- Nur den schmalsten notwendigen Empfangspfad in SpectrumWifiPhy anbinden.
- NSS aus WifiTxVector verwenden und ungültige Antennenkonfigurationen früh ablehnen.
- Beispiel ns3_wifi_sumimo_minimal.cc mit 1x1/2x2-Vergleich anlegen.
- Durchsatz, PER, Stream-SINR und Kanalrang ausgeben.

Abnahmekriterium: 2x2 gewinnt auf gut konditionierten Kanälen und kollabiert auf Rang-1-Kanälen nachvollziehbar gegen 1x1.

PHASE 2: HIGH-LEVEL-API

Geplant ist ein SionnaMimoHelper, der mindestens folgende Aspekte gemeinsam konfiguriert:

- Arraygeometrie: Zeilen, Spalten, Elementabstand und Polarisation
- Ausrichtung: Azimut, Elevation und Rollwinkel
- Betriebsart: SISO, SU-MIMO Spatial Multiplexing, SU-Beamforming, MU-MIMO
- Zahl der TX-/RX-Antennen, maximale NSS und Empfängertyp
- Kanalnormalisierung, Rauschzahl und Effective-SINR-Modell

Die API validiert Wi-Fi-, Sionna- und Abstraktionsparameter zentral, sodass Beispiele keine internen Objekte manuell verdrahten müssen.

PHASE 3: SU-MIMO MIT PRECODING UND BEAMFORMING

Unterphasen:

1. festes, benutzergegebenes Precoding W[k]
2. MRT/MRC für einen Stream
3. SVD-Precoding und optionale Wasserfüllung
4. quantisierte 802.11ax/be-Codebooks und veraltete CSI

Jede Unterphase erhält analytische Referenzkanäle sowie ein Beispiel
ns3_wifi_sumimo_precoding.cc.

  PHASE 4: MU-MIMO

Unterphasen:

1. mehrere CFR-Matrizen H_u[k] in einem gemeinsamen Zeitstempel
2. Downlink-ZF und regularisiertes ZF
3. Inter-User-Interferenz in der Stream-SINR
4. RU/NSS-Zuordnung für HE/EHT und Scheduler-Schnittstelle
5. Uplink-MU-MIMO

Das Phasenbeispiel ns3_wifi_mumimo.cc vergleicht SU-Round-Robin mit MU-ZF bei
unterschiedlicher räumlicher Nutzerkorrelation.

  PHASE 5: KALIBRIERUNG, SKALIERUNG UND REPRODUZIERBARKEIT

  - PER-Kurven gegen Link-Level-Referenzen kalibrieren.
  - Sensitivität gegenüber Antennenabstand, Orientierung, CSI-Alter und
  Raytracing-Samples untersuchen.
  - Cache- und IPC-Speicherbedarf für große N_sc, N_tx und N_rx messen.
  - Deterministische Seeds, Ergebnisformate und CI-Matrix für CPU/GPU festlegen.

4. DATENMODELL DER UNTERPHASE 1.1

Für jeden Subcarrier k ist

$$
  \mathbf{H}[k] \in \mathbb{C}^{N_\mathrm{rx}\times N_\mathrm{tx}}.
$$

Der C++-Container speichert die Werte flach. Der Index für den Eintrag
h[k,r,t] lautet

$$
  i(k,r,t) = (k N_\mathrm{rx} + r)N_\mathrm{tx} + t.
$$

Damit ist k die langsamste und t die schnellste Dimension. Diese Reihenfolge
ist unabhängig von NumPy- oder Eigen-internen Layouts und wird in Unterphase
1.2 Teil des IPC-Vertrags.

5. SU-MIMO-MINIMALMODELL

  5.1 SIGNALMODELL

Ohne Precoding gilt je Subcarrier

$$
  \mathbf{y}[k] =
  \sqrt{\frac{P}{N_s}}\,\mathbf{H}[k]\mathbf{x}[k] + \mathbf{n}[k],
$$

mit N_s = N_tx, E[x x^H] = I und räumlich weißem Rauschen
E[n n^H] = N I. Die Eingangsgröße

$$
  \rho = \frac{P}{N}
$$

ist die gesamte Referenz-SNR vor der Small-Scale-CFR-Gewichtung. Das Modell
verlangt in dieser Unterphase N_rx >= N_tx. Overloaded MIMO wird erst mit einer
expliziten Streamauswahl zugelassen.

5.2 LINEARER MMSE-EMPFÄNGER

Mit alpha = rho/N_s und G[k] = H[k]^H H[k] folgt die
Fehlerkovarianz des linearen MMSE-Detektors:

$$
  \mathbf{E}[k] =
  (\mathbf{I} + \alpha\mathbf{G}[k])^{-1}.
$$

Die Post-Processing-SINR des Streams s ist

$$
  \gamma_s[k] = \frac{1}{E_{s,s}[k]} - 1.
$$

Die Formel berücksichtigt gewünschte Leistung, Inter-Stream-Interferenz und
Rauschverstärkung gemeinsam. Sie ist auch für rangdefiziente H numerisch
definiert, weil I + alpha G positiv definit ist.

5.3 SISO-GRENZFALL

Für N_tx = N_rx = 1 ergibt sich

$$
  E = \frac{1}{1 + \rho |h|^2},
  \qquad
  \gamma = \rho |h|^2.
$$

Damit ist der neue Kern im skalaren Grenzfall exakt kompatibel mit dem
üblichen SISO-SNR-Modell.

5.4 FREQUENZKOMPRESSION

Vor einer MCS-spezifischen Kalibrierung wird je Stream eine
Mutual-Information-erhaltende AWGN-SNR ausgewiesen:

$$
  \gamma_{\mathrm{eff},s} =
  \exp\left(
    \frac{1}{N_\mathrm{sc}}\sum_k \ln(1+\gamma_s[k])
  \right)-1.
$$

Die mittlere ideale spektrale Effizienz über alle Streams ist

$$
  \eta =
  \frac{1}{N_\mathrm{sc}}
  \sum_k\sum_s \log_2(1+\gamma_s[k]).
$$

Diese Größe ist eine Validierungsmetrik, keine unmittelbare
802.11-Datenrate. Coding, Pilot-/Guard-Overhead und PPDU-Dauer werden erst in
den folgenden Unterphasen berücksichtigt.

6. VALIDIERUNG DER UNTERPHASE 1.1

Die Suite sionna enthält:

1. Layouttest eines 2 x 2 x 2-Tensors mit eindeutig nummerierten Koeffizienten.
2. SISO-Test mit h = 2+j und rho=10; erwartet wird gamma=50.
3. Orthogonaler 2x2-Kanal H=I und rho=20; jeder Stream erhält gamma=10 und
   eta=2 log2(11).
4. Zwei diagonale Subcarrier diag(1,2) und diag(2,1) bei rho=8; erwartet werden
   die SINRs (4,16) und (16,4) sowie
   gamma_eff=sqrt((1+4)(1+16))-1.

Alle Vergleiche verwenden eine absolute Toleranz von 1e-10. Die Tests benötigen
weder Sionna, ZMQ noch eine GPU und sind deshalb für schnelle CI-Läufe geeignet.

6.1 AUSGEFÜHRTE CPU-VALIDIERUNG

Am 18. Juli 2026 wurden folgende Prüfungen erfolgreich ausgeführt:

- nativer ns-3-Test sionna: vier von vier Fällen bestanden
- Python-Importtest: Sionna 1.2.1, TensorFlow, Mitsuba, NumPy und ZMQ verfügbar
- bestehende Server-Suite: sieben von sieben Tests in 166,018 s bestanden
- P2P-End-to-End-Test example-sionna-sensing: drei CFR-Antworten, zwei
  empfangene UDP-Pakete zu je 1024 Byte und Cache-Hit-Rate 0,993056

Der ältere Beispielpfad example-sionna ist in der unveränderten Basis defekt:
Er startet standardmäßig Mode 3 mit Submode 1, berechnet aber keinen
Look-ahead-Zeitpunkt. Dadurch wird kein Receiver angelegt und Sionna meldet
ValueError: Scene has no receivers. Der explizite P2P-Beispielpfad ist davon
nicht betroffen. Eine Korrektur gehört in eine eigene kleine Baseline-Subphase.

7. BEKANNTE GRENZEN UND NÄCHSTE ENTSCHEIDUNGSPUNKTE

- Der aktuelle Server liefert weiterhin nur 1x1-CFR; Unterphase 1.2 schließt
  diese Lücke.
- H ist normalisiert. Die genaue MIMO-Normalisierung muss so gewählt werden,
  dass ein Antennenzuwachs nicht versehentlich zusätzliche Gesamtleistung
  erzeugt. Vorgesehen ist eine Normierung über Subcarrier und Antennenpaare mit
  separat erhaltener absoluter Wideband-Leistung.
- Perfekte RX-CSI ist zunächst bewusst idealisiert.
- Pro Antenne wird genau ein Stream angenommen; Streamauswahl und N_s < N_tx
  gehören zur Precoding-Phase.
- Die MI-Kompression ist noch keine kalibrierte HE/EHT-PER-Abbildung.
- Die bestehende Cache-Schlüsselbildung nimmt Reziprozität an. Für
  nicht-reziprokes Precoding müssen Richtung und Konfiguration Teil des
  Schlüssels werden.

8. AUSFÜHRUNG

Vom ns-3.40-Verzeichnis:

```
./ns3 configure --enable-examples --enable-tests
./ns3 build
./test.py -s sionna
```

PDF neu erzeugen:

```
    conda run -n sionna-venv python
    contrib/sionna/tools/render_model_pdf.py
    contrib/sionna/ns3sionna_mimo_model.md
    contrib/sionna/ns3sionna_mimo_model.pdf

Das Rendering-Skript benötigt nur Matplotlib und läuft im vorhandenen
CPU-Environment.
```